

87. Un tableau de « struct »

Trois ennemis, c'est UNE ligne de plus — pas neuf variables.

1. Ce qu'il faut comprendre

`troupe[i].vie` : l'index est calculé à l'exécution, le décalage du champ est connu à la compilation. **Le compilateur fait le calcul d'adresse** ; nous, nous écrivons ce que nous voulons dire.

C'est ici que la `struct` prend son sens. Passer de un à trois ennemis coûte une ligne ; sans elle, il aurait fallu six variables de plus et six lignes à tenir d'accord.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **539 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         178 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← nombre() - trouve l'adresse d'une case de l'écran
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        160 o ← CopierTuiles - 10 lettres sur 43
```

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 1    46 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois ennemis rangés dans un TABLEAU de struct.
// On blesse celui du milieu, puis on affiche les vies des trois : 3 2 3.
//
// Ennemi troupe[3];   = 3 ennemis : troupe[0], troupe[1], troupe[2].
// troupe[i].vie       = la vie de l'ennemi numéro i.

#include <nombre> // écrit un nombre en chiffres
```

2. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {
    uint8_t x, y, vie;           // Trois cases d'un coup, séparées par des virgules.
};
```

3. UN TABLEAU — TROUPE

```
Ennemi troupe[3];               // 3 ennemis.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                    // Le jeu commence ici.
    for (uint8_t i = 0; i < 3; i++) { // Pour chaque ennemi i (0, 1, 2) :
        troupe[i].x = 20 + i * 40;    // x = 20, 60, 100
        troupe[i].y = 60;
        troupe[i].vie = 3;           // 3 vies chacun.
    }

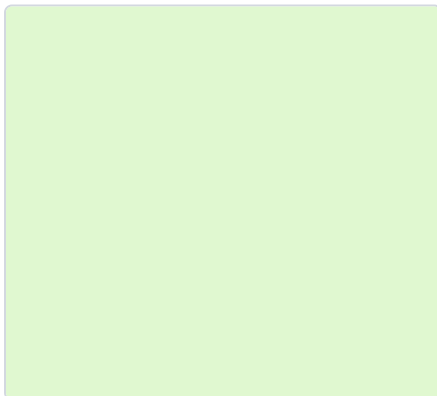
    troupe[1].vie--;              // L'ennemi numéro 1 (celui du milieu) perd une vie : 2.

    for (uint8_t i = 0; i < 3; i++) nombre(4 + i * 5, 8, troupe[i].vie, 1);
                                   // Affiche les 3 vies, en colonnes 4, 9, 14 : 3 2 3.

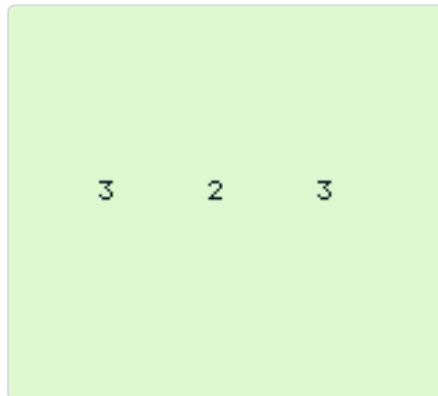
    while (true) {               // Répète pour toujours :
        image();                 // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« 3 2 3 » — seul celui du milieu a été touché.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier est intact
- ✓ celui du milieu a perdu une vie
- ✓ le troisième est intact
- ✓ écrire dans une case n'a pas touché ses voisins

Ensuite : **88. Une méthode : la fonction qui connaît son objet** — chapitre 13 : Ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++